PHYSIOCARE CLINIC

Complete Project Explanation

A code-grounded guide for rebuilding, presenting, and defending the system

React / Vite + Express + Supabase

Generated from the current repository

Prepared as a university project study document

Table of Contents

The page numbers are generated from the final layout. Use the chapter titles as the navigation map for a presentation or code walkthrough.

1: System at a glance
2: Rebuilding from an empty folder
3: Repository structure
4: Runtime architecture and execution flow
5: Backend: server, routes, middleware, utilities
6: Backend services and controllers
7: Frontend: routing, auth, data fetching, UI
8: Clinical assessment feature family
9: Database and migrations
10: API endpoint catalogue
11: Security, validation, and error handling

PHYSIOCARE CLINIC

Complete Project Explanation

A code-grounded guide for rebuilding, presenting, and defending the system

React / Vite + Express + Supabase

Generated from the current repository

Prepared as a university project study document

Table of Contents

The page numbers are generated from the final layout. Use the chapter titles as the navigation map for a presentation or code walkthrough.

1: System at a glance
2: Rebuilding from an empty folder
3: Repository structure
4: Runtime architecture and execution flow
5: Backend: server, routes, middleware, utilities
6: Backend services and controllers
7: Frontend: routing, auth, data fetching, UI
8: Clinical assessment feature family
9: Database and migrations
10: API endpoint catalogue
11: Security, validation, and error handling

12: Testing and quality checks
13: Important programming concepts
14: Project defense questions and answers
15: How to rebuild this project from scratch
16: Final architecture summary
Appendix: Important file inventory

1. System at a Glance

PhysioCare Clinic is a role-aware physiotherapy platform. Patients register, complete a medical intake, browse gender-aware clinician availability, book appointments, and view their recovery journey. Physiotherapists manage availability, appointments, session evaluations, patient clinical profiles, and region-specific assessments. An administrator manages clinicians, patients, assignments, and operational dashboards.

Confirmed technology stack

- Frontend: React 19 with Vite 8, React Router, Axios, i18next/react-i18next, and CSS files organized beside pages and feature families.
- Backend: Node.js ES modules, Express 5, CORS, dotenv, Multer, and the Supabase JavaScript client.
- Data and identity: Supabase Auth plus PostgreSQL tables, row-level security policies, SQL functions, indexes, and private Storage objects.
- Testing: Node built-in test runner for backend behavior and Vitest-style frontend test files consumed by the project tooling.

Architecture diagram

```
User browser
  | React routes and components
  | Axios requests with bearer token
  v
Express app (backend/src/app.js)
  | route middleware -> controller -> service
  | Supabase publishable client or service-role client
  v
Supabase Auth + PostgreSQL + private Storage
  | JSON response / signed URL
  v
React state update -> rerender -> bilingual UI
```

What is deliberately not claimed

The repository proves implementation details, not the original author motivation. Where this guide discusses why a choice is sensible, it labels the reasoning as an engineering rationale inferred from the code.

2. Rebuilding from an Empty Folder

The real development order follows the dependencies between the layers. The database contract and authentication rules must be available before protected backend operations can be reliable, while the frontend can be built against the stable API shapes.

Build sequence

```
Idea / clinic roles
  -> initialize root workspace
  -> create backend and frontend packages
  -> write Supabase migrations in order
  -> configure Auth and environment variables
  -> create Express app, clients, middleware, routes
  -> add domain services and controllers
  -> create React shell, routes, auth context
  -> add dashboards, booking, records, assessments
  -> connect Axios to API and Supabase public config
  -> add validation, RLS, tests, lint, build
  -> run and verify the complete clinic
```

Commands a developer would use

```
mkdir physio && cd physio
npm init -y
mkdir backend frontend supabase
cd backend && npm init -y
npm install express cors dotenv multer @supabase/supabase-js
npm install -D nodemon
cd ../frontend && npm create vite@latest . -- --template react
npm install react-router-dom axios i18next react-i18next @supabase/supabase-js
cd .. && npm install -D concurrently
```

Why each stage exists

Root package: Runs both workspaces with concurrently and documents the install/run workflow.
Migrations: Define profiles, scheduling, records, evaluations, assessments, constraints, indexes, and RLS before code queries them.
Backend config: Loads secrets once and creates public or service-role Supabase clients.
Middleware: Authenticates a bearer token, reloads the active profile, and enforces role boundaries.
Services: Keep scheduling mathematics, validation, auth, and domain rules out of route declarations.
React shell: Owns browser routes, auth state, dashboard composition, forms, bilingual labels, and responsive presentation.

3. Repository Structure

This tree is derived from the current repository. Generated dependency directories and build output are omitted because they are not authored application logic.

```
physio/
|-- package.json                          root scripts
|-- README.md                             setup, routes, checks
|-- backend/
|   |-- package.json                       backend scripts/dependencies
|   |-- src/
|   |   |-- server.js                      process entry point
|   |   |-- app.js                         Express composition
|   |   |-- config/                        Supabase clients
|   |   |-- middleware/                    auth and roles
|   |   |-- routes/                        URL-to-controller mapping
|   |   |-- controllers/                   HTTP orchestration
|   |   |-- services/                      domain rules and assessment validators
|   |   '-- utils/                         response and validation helpers
|   '-- test/                              backend behavior tests
|-- frontend/
|   |-- index.html, vite.config.js         browser/build entry
|   '-- src/
|       |-- main.jsx, App.jsx               React root and route tree
|       |-- auth/                           provider, context, guards
|       |-- services/                       Axios and Supabase session bridge
|       |-- components/                     shared UI primitives
|       |-- pages/                          auth, dashboards, medical flows
|       |-- features/clinical-assessments/  region modules and controls
|       |-- i18n/                            English/Arabic translations
|       '-- assets/                        images and SVG assets
|-- supabase/migrations/                   ordered PostgreSQL/RLS changes
'-- .env files                             ignored local configuration
```

Folder dependency direction

Routes import middleware and controllers. Controllers import services and Supabase clients. Services receive authenticated request context and perform domain work through req.db. React pages import shared components, the API service, auth hooks, and feature configuration. SQL migrations are deployed separately but define the tables and policies every backend query assumes.

4. Runtime Architecture and Execution Flow

The application has two processes started by the root dev script. The backend is an HTTP API on port 5000 by default; Vite serves the React application on port 5173 unless that port is occupied.

Backend startup: actual code

```
  1 | import dotenv from 'dotenv';
  2 |
  3 | dotenv.config();
  4 |
  5 | const { default: app } = await import('./app.js');
  6 |
  7 | const PORT = process.env.PORT || 5000;
  8 |
  9 | app.listen(PORT, () => {
 10 |   console.log('Backend server running on http://localhost:${PORT}');
 11 | });
 12 |
```

Line 1 imports dotenv using an ES module default import. dotenv.config() reads backend/.env when the process working directory is backend. The dynamic import waits until environment variables exist before loading app.js; this matters because config/supabase.js validates SUPABASE_URL and the public key during module initialization. The port uses process.env.PORT with 5000 as a fallback, then app.listen keeps the Node process serving requests.

Express composition: actual code

```
 1 | import express from 'express';
 2 | import cors from 'cors';
 3 | import authRoutes from './routes/authRoutes.js';
 4 | import testRoutes from './routes/testRoutes.js';
 5 | import adminRoutes from './routes/adminRoutes.js';
 6 | import physiotherapistRoutes from './routes/physiotherapistRoutes.js';
 7 | import medicalRecordRoutes from './routes/medicalRecordRoutes.js';
 8 | import { appointmentRoutes, physiotherapistDirectory, profileRoutes } from './routes/clinicRoutes.
 9 |
10 | const app = express();
11 |
12 | app.use(
13 |   cors({
14 |     origin: process.env.FRONTEND_URL || 'http://localhost:5173',
15 |     credentials: true,
16 |   }),
17 | );
18 |
19 | app.use(express.json());
20 | app.use(express.urlencoded({ extended: true }));
21 |
22 | app.get('/', (req, res) => {
23 |   res.json({
24 |     message: 'Physiotherapy Clinic API',
25 |   });
26 | });
27 |
28 | app.use('/api/auth', authRoutes);
29 | app.use('/api/test', testRoutes);
30 | app.use('/api/admin', adminRoutes);
31 | app.use('/api/physiotherapists', physiotherapistDirectory);
32 | app.use('/api/appointments', appointmentRoutes);
33 | app.use('/api/physiotherapist', physiotherapistRoutes);
34 | app.use('/api/profile', profileRoutes);
35 | app.use('/api/medical-records', medicalRecordRoutes);
36 |
```

Express creates the app object, CORS is configured with the local frontend origin and credentials, JSON/urlencoded parsers decode request bodies, and each /api prefix mounts a router. The final middleware returns a consistent JSON 404 rather than an HTML page.

Frontend startup

```
  1 | import { StrictMode } from 'react'
  2 | import { createRoot } from 'react-dom/client'
  3 | import { BrowserRouter } from 'react-router-dom'
  4 | import './i18n/index.js'
  5 | import './index.css'
  6 | import App from './App.jsx'
  7 | import { AuthProvider } from './auth/AuthContext.jsx'
  8 |
  9 | createRoot(document.getElementById('root')).render(
 10 |   <StrictMode>
 11 |     <BrowserRouter>
 12 |       <AuthProvider>
 13 |         <App />
 14 |       </AuthProvider>
 15 |     </BrowserRouter>
 16 |   </StrictMode>,
 17 | )
 18 |
```

createRoot attaches React to the #root element. StrictMode exposes unsafe side effects during development; BrowserRouter supplies client-side navigation; AuthProvider initializes Supabase session state before protected RoleRoute pages render.

One request end-to-end

```
Patient clicks Book
  -> PatientDashboard handler
  -> Axios POST /api/appointments
  -> appointmentRoutes auth + patient role middleware
  -> clinicController.bookAppointment
  -> availabilityService + Supabase transaction-like checks
  -> JSON success or conflict/error
  -> component state update
  -> calendar / appointment list rerenders
```

5. Backend: Server, Routes, Middleware, Utilities

The backend is intentionally layered. URL declarations stay small; controllers translate HTTP into domain calls; services enforce rules that should also be testable without a running server.

Supabase configuration

```
 1 | import { createClient } from '@supabase/supabase-js';
 2 |
 3 | const supabaseUrl =
 4 |   process.env.SUPABASE_URL || process.env.NEXT_PUBLIC_SUPABASE_URL;
 5 | const supabaseKey =
 6 |   process.env.SUPABASE_PUBLISHABLE_KEY ||
 7 |   process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
 8 | const serviceRoleKey =
 9 |   process.env.SUPABASE_SERVICE_ROLE_KEY || process.env.SUPABASE_SECRET_KEY;
10 |
11 | if (!supabaseUrl || !supabaseKey) {
12 |   throw new Error(
13 |     'SUPABASE_URL and SUPABASE_PUBLISHABLE_KEY must be set in backend/.env',
14 |   );
15 | }
16 |
17 | export function createSupabaseClient() {
18 |   return createClient(supabaseUrl, supabaseKey, {
19 |     auth: {
20 |       persistSession: false,
21 |       autoRefreshToken: false,
22 |     },
23 |   });
24 | }
25 |
26 | export function getSupabasePublicConfig() {
27 |   return {
28 |     url: supabaseUrl,
29 |     publishableKey: supabaseKey,
30 |   };
31 | }
32 |
33 | export function createAdminClient() {
34 |   if (!serviceRoleKey) {
35 |     throw new Error(
36 |       'SUPABASE_SERVICE_ROLE_KEY or SUPABASE_SECRET_KEY must be set in backend/.env',
37 |     );
38 |   }
39 |
40 |   return createClient(supabaseUrl, serviceRoleKey, {
41 |     auth: { persistSession: false, autoRefreshToken: false },
42 |   });
43 | }
44 |
45 | const supabase = createSupabaseClient();
46 |
47 | export default supabase;
48 |
```

The module reads SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, and either SUPABASE_SERVICE_ROLE_KEY or SUPABASE_SECRET_KEY. createSupabaseClient creates a non-persistent public client for user-authenticated operations. createAdminClient creates the backend-only client used after requireAuth reloads a profile. The startup guard prevents a misleading server that cannot authenticate or query anything.

Authentication middleware

```
import { createAdminClient, createSupabaseClient } from '../config/supabase.js';

export async function requireAuth(req, res, next) {
  const authorization = req.get('authorization') || '';
  const token = authorization.startsWith('Bearer ') ? authorization.slice(7) : '';
  if (!token) return res.status(401).json({ success: false, message: 'Authentication required', er

  try {
    const authClient = createSupabaseClient();
    const { data: { user }, error } = await authClient.auth.getUser(token);
    if (error || !user) return res.status(401).json({ success: false, message: 'Invalid or expired

    const admin = createAdminClient();
    const { data: profile, error: profileError } = await admin
      .from('profiles')
      .select('id, first_name, last_name, email, phone, gender, date_of_birth, role, is_active')
      .eq('id', user.id)
      .single();
    if (profileError || !profile) return res.status(401).json({ success: false, message: 'Account
    if (!profile.is_active) return res.status(403).json({ success: false, message: 'Account is dis

    req.auth = { token, user, profile };
    req.db = admin;
    next();
  } catch (error) {
    console.error(error);
    const missingServerKey = error.message?.includes('must be set in backend/.env');
    return res.status(missingServerKey ? 503 : 500).json({
      success: false,
      message: missingServerKey
        ? 'Backend Supabase secret key is not configured'
        : 'Unable to authenticate request',
      errors: [],
    });
  }
}

```

requireAuth extracts only a Bearer token, rejects missing tokens with 401, validates the token through Supabase Auth, then uses the admin client to select the active profile. The profile is placed on req.auth and the admin client on req.db. This reload is important: a stale browser role is not trusted. requireRole is a second, explicit gate that returns 403 when the authenticated profile role is not allowed.

HTTP utility contract

```
export function apiError(message, statusCode = 400, errors = []) {
  const error = new Error(message);
  error.statusCode = statusCode;
  error.errors = errors;
  return error;
}

export function sendError(res, error, fallback = 'Operation failed') {
  const status = error.statusCode || 500;
  if (status >= 500) console.error(error);
  return res.status(status).json({
    success: false,
    message: status >= 500 ? fallback : error.message,
    errors: error.errors || [],
  });
}

export function ok(res, data = {}, message = 'Operation completed successfully', status = 200) {
  return res.status(status).json({ success: true, message, data });
}

export function required(body, fields) {
  const missing = fields.filter((field) => body[field] === undefined || body[field] === null || St
  if (missing.length) throw apiError('Required fields are missing', 400, missing.map((field) => ({
}

export function pick(source, allowed) {
  return Object.fromEntries(allowed.filter((key) => source[key] !== undefined).map((key) => [key,
}

```

apiError: Creates an Error with an HTTP status and structured field errors.
sendError: Converts thrown errors into { success, message, errors }, logging 5xx failures.
ok: Creates consistent successful JSON responses with data and message.
required: Rejects undefined, null, and blank fields and reports each missing field.
pick: Whitelists update fields so clients cannot mass-assign protected columns.

Route map

/api/auth: Public config, sign-in, sign-up.
/api/test: Supabase connectivity test.
/api/admin: Admin-only dashboard, users, clinicians, assignments.
/api/physiotherapists: Authenticated directory and available slots.
/api/appointments: Patient-only availability, booking, own appointments, recovery, cancel/reschedule.
/api/profile: Authenticated own profile GET/PATCH.
/api/physiotherapist: Clinician profile, hours, time off, appointments, patients, assessments.
/api/medical-records: Patient record, documents, signed URLs, authorized clinician/admin view.

6. Backend Services and Controllers

Controllers are the HTTP boundary; services contain rules that are reused by controllers and tests. The following are the important flows confirmed by imports and tests.

Authentication flow

```
POST /api/auth/signin
  authController.signIn validates email/password types
  -> authService.signInUser normalizes email
  -> Supabase signInWithPassword
  -> load profiles row and active flag
  -> return user, session, profile
  -> frontend stores session through establishSession
```

signUpUser rejects the administrator email, stores first/last name and gender in Auth metadata, and upserts a profile when Supabase returns an immediate session. signInUser distinguishes unconfirmed email, invalid credentials, missing migration tables, inactive profiles, and generic failures.

Availability and booking

```
import { apiError } from '../utils/http.js';

export const CLINIC_TIME_ZONE = 'UTC';
export const WEEKDAYS = [
  'sunday',
  'monday',
  'tuesday',
  'wednesday',
  'thursday',
  'friday',
  'saturday',
];

const DATE_PATTERN = /^\d{4}-\d{2}-\d{2}$/;
const ACTIVE_APPOINTMENT_STATUSES = ['pending', 'confirmed'];

export function isIsoDate(value) {
  if (!DATE_PATTERN.test(String(value || ''))) return false;
  const parsed = new Date(`${value}T00:00:00.000Z`);
  return !Number.isNaN(parsed.getTime()) && parsed.toISOString().slice(0, 10) === value;
}

export function addUtcDays(date, amount) {
  const parsed = new Date(`${date}T00:00:00.000Z`);
  parsed.setUTCDate(parsed.getUTCDate() + amount);
  return parsed.toISOString().slice(0, 10);
}

export function inclusiveDayCount(from, to) {
  const start = new Date(`${from}T00:00:00.000Z`);
  const end = new Date(`${to}T00:00:00.000Z`);
  return Math.floor((end - start) / 86400000) + 1;
}

function slotIso(date, time) {
  return new Date(`${date}T${String(time).slice(0, 8)}Z`).toISOString();
}

function overlaps(start, end, busyStart, busyEnd) {
  return start < busyEnd && end > busyStart;
}

function clinicianDirectoryRecord(clinician) {
  return {
    profile_id: clinician.profile_id,
    professional_title: clinician.professional_title,
    specialization: clinician.specialization,
    biography: clinician.biography,
    years_of_experience: clinician.years_of_experience,
    consultation_duration: clinician.consultation_duration,
    profile_image: clinician.profile_image,
    is_accepting_patients: clinician.is_accepting_patients,
    profiles: clinician.profiles,
  };
}

export async function eligibleClinicians(req, requestedId, options = {}) {
  const requirePatientGender = options.requirePatientGender ?? false;
  const patientGender = req.auth.profile.role === 'patient'
    ? req.auth.profile.gender
```

isIsoDate validates both shape and calendar reality. addUtcDays and inclusiveDayCount support bounded UTC ranges. eligibleClinicians joins clinician profiles, filters accepting clinicians and active profiles, and applies the female-patient rule. clinicianSlots fetches working hours, active appointments, and time-off in parallel; it builds per-clinician busy periods, emits future slots that fit completely in a period, removes overlaps, and sorts deterministically. groupSlotsByDay merges clinicians offering the same start time while retaining doctor_ids and available_doctor_count. shuffled is a Fisher-Yates copy operation used when any eligible clinician may be selected.

Controller responsibilities

adminController.js: Admin dashboard summaries, clinician CRUD/disable, patient management, assignments.
clinicController.js: Directory, availability, booking, reschedule/cancel, profile, recovery journey.
physiotherapistController.js: Clinician profile, working hours, time off, appointment transitions, patient lists.
medicalRecordController.js: Medical intake persistence, documents, signed URLs, authorized read-only record view.
clinicalAssessmentController.js: Assessment creation/update/reassessment, region selection, completion and clinician access.

Assessment validation family

clinicalAssessmentService owns metadata rules and the shoulder schema; dedicated services validate elbow, cervical, lumbar, wrist, hip, knee, and ankle payloads. The registry-style frontend modules mirror this backend split. Completed assessments require the defined sections and an affected side where the metadata rules demand it. Unknown fields, unsupported tests, invalid ranges, and oversized payloads are rejected before database writes.

Medical documents

medicalRecordRoutes configures Multer memoryStorage with one file and an 8 MB limit. The upload adapter converts Multer errors to 413 for oversized files or 400 for invalid uploads. The controller stores metadata and uses private Storage paths; documentUrl returns a signed URL rather than exposing a public bucket.

7. Frontend: Routing, Auth, Data Fetching, and UI

The frontend is a route-driven React single-page application. Shared auth and services keep pages focused on presentation and user interaction.

Route tree

```
 1 | import { Navigate, Route, Routes } from 'react-router-dom'
 2 | import Home from './pages/Home.jsx'
 3 | import SignIn from './pages/auth/SignIn.jsx'
 4 | import Register from './pages/auth/Register.jsx'
 5 | import ConnectionTest from './pages/ConnectionTest.jsx'
 6 | import PatientDashboard from './pages/dashboards/PatientDashboard.jsx'
 7 | import PhysiotherapistDashboard from './pages/dashboards/PhysiotherapistDashboard.jsx'
 8 | import AdminDashboard from './pages/dashboards/AdminDashboard.jsx'
 9 | import MedicalIntake from './pages/medical/MedicalIntake.jsx'
10 | import MedicalRecordViewer from './pages/medical/MedicalRecordViewer.jsx'
11 | import PatientClinicalProfile from './pages/clinical/PatientClinicalProfile.jsx'
12 | import ShoulderAssessment from './features/clinical-assessments/shoulder/ShoulderAssessment.jsx'
13 | import ElbowAssessment from './features/clinical-assessments/elbow/ElbowAssessment.jsx'
14 | import NeckAssessment from './features/clinical-assessments/neck/NeckAssessment.jsx'
15 | import LumbarAssessment from './features/clinical-assessments/lumbar/LumbarAssessment.jsx'
16 | import WristAssessment from './features/clinical-assessments/wrist/WristAssessment.jsx'
17 | import HipAssessment from './features/clinical-assessments/hip/HipAssessment.jsx'
18 | import KneeAssessment from './features/clinical-assessments/knee/KneeAssessment.jsx'
19 | import AnkleAssessment from './features/clinical-assessments/ankle/AnkleAssessment.jsx'
20 | import RoleRoute from './components/RoleRoute.jsx'
21 | import './pages/dashboards/Dashboard.css'
22 |
23 | function App() {
24 |   return (
25 |     <Routes>
26 |       <Route path="/" element={<Home />} />
27 |       <Route path="/signin" element={<SignIn />} />
28 |       <Route path="/register" element={<Register />} />
29 |       <Route path="/connection-test" element={<ConnectionTest />} />
30 |       <Route path="/patient/dashboard" element={<RoleRoute allowedRoles={['patient']}><PatientDash
31 |       <Route path="/patient/medical-profile" element={<RoleRoute allowedRoles={['patient']}><Medic
32 |       <Route path="/medical-records/patient/:patientId" element={<RoleRoute allowedRoles={['physio
33 |       <Route path="/physiotherapist/patients/:patientId" element={<RoleRoute allowedRoles={['physi
34 |       <Route path="/physiotherapist/patients/:patientId/assessments/elbow/:assessmentId" element={
35 |       <Route path="/physiotherapist/patients/:patientId/assessments/cervical/:assessmentId" elemen
36 |       <Route path="/physiotherapist/patients/:patientId/assessments/lumbar/:assessmentId" element=
37 |       <Route path="/physiotherapist/patients/:patientId/assessments/wrist/:assessmentId" element={
38 |       <Route path="/physiotherapist/patients/:patientId/assessments/hip/:assessmentId" element={<R
39 |       <Route path="/physiotherapist/patients/:patientId/assessments/knee/:assessmentId" element={<
40 |       <Route path="/physiotherapist/patients/:patientId/assessments/ankle/:assessmentId" element={
41 |       <Route path="/physiotherapist/patients/:patientId/assessments/:assessmentId" element={<RoleR
42 |       <Route
```

Public pages are Home, SignIn, Register, and ConnectionTest. RoleRoute wraps patient, physiotherapist, and admin pages. Assessment routes include a generic shoulder fallback plus explicit elbow, cervical, lumbar, wrist, hip, knee, and ankle paths. A wildcard redirects unknown URLs to Home.

AuthProvider state machine

```
import {
  useCallback,
  useEffect,
  useMemo,
  useRef,
  useState,
} from 'react'
import AuthContext from './context.js'
import api from '../services/api.js'
import {
  getSupabaseClient,
  migrateLegacySession,
  setAuthSessionSnapshot,
} from '../services/supabase.js'

function readCachedProfile(userId) {
  try {
    const cached = JSON.parse(localStorage.getItem('user_profile') || 'null')
    return cached?.id === userId && cached?.role ? cached : null
  } catch {
    return null
  }
}

export function AuthProvider({ children }) {
  const [session, setSession] = useState(null)
  const [user, setUser] = useState(null)
  const [profile, setProfile] = useState(null)
  const [loading, setLoading] = useState(true)
  const profileRef = useRef(null)
  const syncVersionRef = useRef(0)

  const storeProfile = useCallback((nextProfile) => {
    profileRef.current = nextProfile || null
    setProfile(nextProfile || null)
    if (nextProfile) localStorage.setItem('user_profile', JSON.stringify(nextProfile))
    else localStorage.removeItem('user_profile')
  }, [])

  const syncSession = useCallback(async (nextSession, suppliedProfile = null) => {
    const syncVersion = ++syncVersionRef.current
    setAuthSessionSnapshot(nextSession)
    setSession(nextSession || null)
    setUser(nextSession?.user || null)

    if (!nextSession?.user) {
      storeProfile(null)
      setLoading(false)
      return
    }

    let nextProfile = suppliedProfile
    if (!nextProfile && profileRef.current?.id === nextSession.user.id) {
      nextProfile = profileRef.current
    }

    if (!nextProfile) {
      try {
        const response = await api.get('/profile/me')
        nextProfile = response.data.data
```

AuthProvider owns session, user, profile, and loading state. initializeAuth loads the public Supabase configuration, restores an existing session, bridges the legacy storage key once, and subscribes to Supabase auth changes. syncSession updates the token snapshot, loads /profile/me when needed, guards against stale async responses with syncVersionRef, and persists a safe profile cache. establishSession calls Supabase setSession after the backend sign-in response.

Axios and API URL normalization

```
import axios from 'axios'
import { getAuthAccessToken, getSupabaseClient } from './supabase.js'

const configuredUrl = import.meta.env.VITE_API_URL || 'http://localhost:5000'
const normalizedUrl = configuredUrl.replace(/\/$/, '')
const API_URL = normalizedUrl.endsWith('/api')
  ? normalizedUrl
  : `${normalizedUrl}/api`

const api = axios.create({
  baseURL: API_URL,
  headers: {
    'Content-Type': 'application/json',
  },
})

api.interceptors.request.use((config) => {
  const accessToken = getAuthAccessToken()
  if (accessToken) config.headers.Authorization = `Bearer ${accessToken}`
  return config
})

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401 && !error.config?.url?.includes('/auth/')) {
      localStorage.removeItem('user_profile')
      void getSupabaseClient().then((client) => client.auth.signOut())
    }
    return Promise.reject(error)
  },
)

export async function testBackendConnection() {
  const response = await api.get('/test')
  return response.data
}

export default api
```

The API service accepts either http://localhost:5000 or a full /api URL and normalizes it to exactly one /api suffix. The interceptor adds the current access token and clears cached auth after protected 401 responses. testBackendConnection powers the sign-in database status banner.

UI organization

- components/ contains reusable Brand, LanguageSwitcher, BookingInstructions, RoleRoute, and assessment controls.
- pages/auth handles sign-in and registration; pages/dashboards contains three role dashboards, layout, recovery, and session evaluation.
- pages/medical contains the multi-step intake, body chart, pain-area picker, and read-only medical viewer.
- features/clinical-assessments uses one folder per body region with data defaults, labels/config, sections, and a shared control vocabulary.
- i18n/locales/en and i18n/locales/ar keep UI copy separate from logic and sync document direction/language.

Sign-in event flow

```
submit form
  -> handleSubmit prevents browser navigation
  -> setLoading(true), disable inputs
  -> api.post(/auth/signin, normalized email + password)
  -> map HTTP/message to a translation key
  -> establishSession(session, profile)
  -> navigate by profile.role
  -> finally setLoading(false)
```

8. Clinical Assessment Feature Family

Clinical assessments are deliberately modeled as region-specific schemas rather than one unrestricted JSON form. This gives each body region its own vocabulary while preserving a common lifecycle.

Shared lifecycle

```
Clinician opens patient profile
  -> POST assessment metadata (region/type/version)
  -> backend verifies assignment/appointment access
  -> frontend loads region data defaults
  -> section components update nested state
  -> PATCH draft with affected_side and assessment_data
  -> optional autosave after debounce
  -> final completion validates every required section
  -> reassessment creates parent-linked record
```

Region coverage found in source

Shoulder: Nine structured sections: observation, palpation, articular, muscular, neurological, functional, special tests, reasoning, outcome.
Elbow: Physical, strength, neurological, and final sections with targeted tests and ranges.
Cervical: Thirteen-section schema with neurological/radicular safety findings and pathways.
Lumbar: Twelve sections with pathways, tests, red flags, measurements, and completion rules.
Wrist: Physical, muscular, neurological, functional, special tests, and final sections.
Hip, knee, ankle: Dedicated data/config/section modules; backend validates bounded nested shapes and payload size.

Why the shared controls matter

AssessmentControls centralizes bilingual labels, choice buttons, checkboxes, notes, number

fields, finding cards, and section intros. A section component describes the clinical fields; the controls standardize input behavior and visual language. This lowers duplication while preserving region-specific data keys.

Clinical safety boundaries

- The backend validates ranges and known selections but does not generate a diagnosis.
- Role and ownership checks happen before a clinician can read or mutate a patient assessment.
- Final submission requires red-flag answers and complete required sections where the region service defines them.
- Reassessment preserves parent_assessment_id so progress can be compared without overwriting history.

9. Database and Migrations

Supabase PostgreSQL is the system of record. The migration filenames form a chronological contract: run them in order in the Supabase SQL editor or migration workflow.

Migration sequence

202607210001: profiles: identity, role, active state, gender, self-service RLS.
202607220001: physiotherapists, assignments, working hours, time off, appointments, status history, policies/functions.
202607220002: single administrator reconciliation.
202607220003: physiotherapist schema reconciliation.
202607230001: gender-aware directory and booking policies.
202608050001: medical profiles, surgeries, medications, documents, versions, private Storage policies.
202608150001: session evaluations and supporting indexes/policies.
202608150002: clinical assessments, parent links, region/type/version, access policies.
202608180001-003: elbow, cervical, lumbar clinical assessment support.
202608220001-004: wrist, hip/pelvis, knee/thigh, ankle/foot/toes support.

Core relationships

```
auth.users 1 --- 1 profiles
profiles 1 --- 0..1 physiotherapists
profiles(patient) 1 --- many appointments
profiles(physio) 1 --- many appointments
patient <-> physiotherapist through assignments
physiotherapist 1 --- many working_hours / time_off
appointment 1 --- many status_history / evaluations
patient 1 --- 1 medical_profile --- many surgeries, medications, documents, versions
patient + clinician + appointment --- many clinical_assessments
clinical_assessment 1 --- many reassessments via parent_assessment_id
```

RLS and server-side defense in depth

Policies scope profile reads, clinician visibility, assignments, appointment participants, medical records, documents, evaluations, and assessments. The backend adds a second layer by authenticating the bearer token, loading the active profile with the service client, applying role middleware, and constraining controller queries by ownership or assignment. A service key is never placed in the frontend.

Representative SQL contract

```
from auth.users u
where p.id = u.id
  and lower(u.email) = 'omarmassoud27076@gmail.com';

create table if not exists public.physiotherapists (
  id uuid primary key default gen_random_uuid(),
  profile_id uuid not null unique references public.profiles(id) on delete restrict,
  professional_title text not null,
  license_number text not null unique,
  specialization text not null,
  biography text not null default '',
  years_of_experience integer not null default 0 check (years_of_experience >= 0),
  consultation_duration integer not null default 30 check (consultation_duration > 0),
  profile_image text,
  is_accepting_patients boolean not null default true,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

-- Compatibility for projects that already had an older physiotherapists table.
alter table public.physiotherapists
  add column if not exists consultation_duration integer not null default 30
    check (consultation_duration > 0),
  add column if not exists profile_image text;

create table if not exists public.patient_physiotherapist_assignments (
  id uuid primary key default gen_random_uuid(),
  patient_id uuid not null references public.profiles(id) on delete restrict,
```

10. API Endpoint Catalogue

The following map is derived from the route modules. Protected endpoints require Authorization: Bearer <access token>; role restrictions are shown where present.

Authentication and health

GET /api/auth/config: Public Supabase URL/publishable key for frontend initialization.
POST /api/auth/signin: Email/password -> session and profile; 401 invalid credentials, 403 unconfirmed, 503 missing DB setup.
POST /api/auth/signup: Creates patient Auth/profile data; validates name, gender, email, password.
GET /api/test: Supabase connectivity probe; returns database.connected or 503.

Patient and clinic operations

GET /api/physiotherapists: Authenticated clinician directory; gender-aware filtering.
GET /api/physiotherapists/:id: One directory record.
GET /api/physiotherapists/:id/available-slots: Computed future slots for a clinician.
GET /api/appointments/availability: Patient calendar grouped by day.
POST /api/appointments: Patient books a slot; race/conflict checks return 409.
GET /api/appointments/my: Patient appointments.
GET /api/appointments/recovery: Patient recovery journey data.
PATCH /api/appointments/:id/cancel: Patient cancels own appointment.

PATCH /api/appointments/:id/reschedule: Patient reschedules with availability checks.
GET/PATCH /api/profile/me: Authenticated user profile read/update.

Physiotherapist operations

GET/PATCH /api/physiotherapist/me: Clinician professional profile.
GET/POST/PATCH/DELETE /api/physiotherapist/availability...: Working hours CRUD.
GET/POST/DELETE /api/physiotherapist/time-off...: Time-off CRUD.
GET /api/physiotherapist/appointments: Clinician appointments.
PATCH /api/physiotherapist/appointments/:id/status: Pending -> confirmed -> status transitions.
POST /api/physiotherapist/appointments/:id/complete: Completes with evaluation workflow.
GET /api/physiotherapist/patients...: Assigned/booked patient access and clinical profile.
POST/PATCH/GET .../assessments: Create, update, retrieve, and reassess region assessments.

Admin and medical records

GET/POST/PATCH/DELETE /api/admin/*: Admin dashboard, clinician/patient CRUD, assignments and disables.
GET/PUT /api/medical-records/me: Patient medical intake read/save.
POST /api/medical-records/documents: Patient upload; one file, 8 MB maximum.
DELETE /api/medical-records/documents/:id: Patient removes own document.
GET /api/medical-records/documents/:id/url: Signed private document URL.
GET /api/medical-records/patient/:patientId: Assigned clinician/admin read-only medical record.

11. Security, Validation, and Error Handling

The code treats clinical data and role boundaries as security-sensitive. Validation is performed at multiple boundaries so a friendly browser form is not the only defense.

Authentication and authorization

- The frontend keeps a Supabase session and sends its access token through Axios.
- requireAuth verifies the token with Supabase Auth and reloads the active profile server-side.
- requireRole prevents a valid patient from calling admin or clinician routers.
- Controllers constrain resources by patient_id, physiotherapist_id, assignment, or appointment participation.
- Supabase RLS remains an independent database-level isolation layer.

Validation patterns

required/pick: Prevent missing fields and mass assignment.
apiError/sendError: Preserve status and structured errors at the HTTP boundary.
Assessment knownKeys: Reject unknown JSON keys and unsupported tests.
Range/enum checks: Reject impossible pain, strength, ROM, dates, and selections.
Multer limits: Reject oversized or malformed document uploads.
UTC slot math: Avoid local-time ambiguity for booking overlaps.

Failure behavior users can observe

- Network/config failure: sign-in and registration show a service/network translation key.
- 401: token/session or credentials are invalid; protected cached profile is cleared by Axios.
- 403: authenticated role or account is not authorized/active.
- 409: booking conflict, duplicate assignment, missing gender prerequisite, or race condition.

- 503: Supabase/database schema is unavailable or required migration is missing.
- 404: unknown API route or concealed resource ownership.

Important security note

Never commit backend/.env, especially SUPABASE_SECRET_KEY or a service-role key. The repository ignores both .env files. If a secret is exposed in a message, screenshot, commit, or log, rotate it in Supabase and update only the local ignored file.

12. Testing and Quality Checks

The repository includes behavior-focused backend tests and small frontend data tests. The commands below are the checks documented in README.md and executed during this analysis.

Backend tests

availabilityService.test.js: Dates, aligned slots, booked periods, grouping, shuffling, gender filtering, race retry.
bookingController.test.js: Medical profile prerequisite and concurrent booking conflict behavior.
clinicalAssessmentAuthorization.test.js: Assignment/appointment access and patient isolation.
clinicalAssessmentService.test.js: Shoulder metadata/data schemas, ranges, completion rules, safety fields.
elbow/lumbar/neckAssessmentService.test.js: Region schemas, pathways, tests, ranges, complete sections.
medicalRecordController.test.js: Medical record mapping, BMI, age, red flags, durations.
sessionEvaluationController.test.js: Metrics, integer/range validation, notes and comparisons.

Frontend tests and commands

```
npm test --prefix backend
npm run build --prefix frontend
npm run lint --prefix frontend
```

The build validates Vite module resolution and production bundling. Oxlint catches unused variables and other static issues. Tests isolate deterministic domain rules without requiring a browser login. A true end-to-end login test needs a valid Supabase account and should be run deliberately, never with credentials copied into logs.

13. Important Programming Concepts

These are the concepts a student should be able to explain with a concrete file from this project.

ES modules: import/export connect server, services, routes, and React modules; package.json sets type=module.
Async/await: Supabase and Axios calls return Promises; try/catch/finally controls loading and failures.
React components: Functions such as SignIn, dashboards, assessment sections, and shared controls return UI.
Props/state: Pages receive route/context data; useState stores form, loading, error, step, and domain state.
Hooks: useEffect starts initialization/fetches, useCallback stabilizes handlers, useMemo derives values, useRef tracks async versions.
REST: HTTP verbs express read/create/update/delete semantics under /api routes.
Middleware: Express functions authenticate, authorize, parse, or transform requests before controllers.
CRUD: Admin, profile, availability, time-off, appointments, records, documents, and assessments all have CRUD-like flows.
RLS: PostgreSQL policies evaluate auth.uid() and role/access functions for row-level isolation.
Whitelisting: pick and knownKeys accept only explicit fields, limiting accidental or malicious updates.
Debounced autosave: assessment pages delay draft PATCH calls and use refs to prevent overlapping saves.
Internationalization: i18next keys supply English/Arabic labels and the document direction is synchronized.

Representative line-by-line explanation

```
const app = express();
```

const declares a binding that is not reassigned. express is the imported factory function. Calling express() creates the application object that owns middleware, routes, and the HTTP request pipeline. Later app.use, app.get, and app.listen calls all operate on this object.

```
const response = await api.post('/auth/signin', { email, password });
```

const stores the eventual Axios response. await pauses this async handler until the Promise resolves or rejects. api.post selects HTTP POST because credentials are submitted to create a session rather than retrieve a public resource. The object becomes JSON in the request body; a rejected Promise enters catch and maps to a translated user message.

```
if (!token) return res.status(401).json({ success: false, message: 'Authentication required' });
```

The guard checks the boundary before any database work. status(401) communicates that authentication is missing, json sends a machine-readable response, and return stops execution so next middleware cannot run without a token.

14. Project Defense Questions and Answers

Use these answers as speaking prompts. They are tied to actual files and behavior, while inferred motivations are framed as engineering rationale.

Question: Why use both a public Supabase client and an admin client?

Answer: The public client is appropriate for user-authenticated Auth operations. The admin/service client is backend-only and lets requireAuth reload profiles and controllers perform controlled server operations. Keeping the secret out of the browser limits exposure.

Question: Why is the profile reloaded after validating a token?

Answer: A JWT alone may contain stale role or active-state assumptions. requireAuth queries profiles and checks is_active, so authorization uses current database state.

Question: Why are booking slots calculated in a service?

Answer: Slot generation combines hours, appointments, time-off, UTC dates, durations, and overlaps. Keeping it in availabilityService makes the rule reusable and directly testable.

Question: How is a double booking prevented?

Answer: The controller computes eligibility and handles the database conflict/race path; tests explicitly cover a slot lost to a concurrent booking and return a clear 409 response.

Question: Why does the frontend have one folder per assessment region?

Answer: Regions have different fields and clinical tests. Separate data/config/sections prevent a single permissive form from accepting invalid region vocabulary while shared controls keep the UI consistent.

Question: Why use PATCH for drafts?

Answer: PATCH represents partial modification. Assessment pages autosave drafts and update only the current aggregate without pretending to replace an unrelated resource.

Question: How are patients prevented from seeing other patients?

Answer: Backend ownership checks, role middleware, clinician assignment/appointment checks,

and Supabase RLS all constrain reads. Tests assert concealment of unrelated profiles.

Question: Why are files uploaded through memoryStorage?

Answer: The route enforces a single 8 MB file and the controller can validate/store bytes through Supabase private Storage without creating a public filesystem directory.

Question: What happens if a migration is missing?

Answer: Profile lookup or health queries return a schema error; controllers convert it to a 503 message instructing the operator to run migrations.

Question: Why does the frontend use an AuthProvider?

Answer: Many routes need the same session/profile/loading state. A provider avoids prop drilling and lets RoleRoute consistently gate dashboards.

Question: What would you improve for production?

Answer: Add structured logging/observability, automated migration deployment, stronger integration tests, rate limiting, secret rotation procedures, bundle code-splitting, and a centralized error boundary. These are recommendations, not current claims.

15. How to Rebuild This Project From Scratch

Memorize the dependency order, then implement one vertical slice before expanding to every role and body region.

```
1. Initialize root package and workspace scripts.
2. Create backend package and install Express, Supabase, dotenv, CORS, Multer, nodemon.
3. Create frontend Vite package and install React Router, Axios, i18next, Supabase.
4. Add ignored backend/.env and frontend/.env; never commit secrets.
5. Apply migrations 202607210001 through 202608220004 in order.
6. Implement Supabase config and server entry.
7. Implement app middleware and health/auth routes.
8. Implement requireAuth/requireRole and HTTP response helpers.
9. Implement profile, clinician, scheduling, booking, and admin controllers/services.
10. Implement medical records, private documents, evaluations, and assessment validators.
11. Create React root, BrowserRouter, AuthProvider, API/Supabase services.
12. Build public auth pages and RoleRoute-protected dashboards.
13. Add medical intake/body chart and recovery flows.
14. Add shared assessment controls and one region at a time.
15. Connect forms to endpoints with loading/error/success states.
16. Add bilingual translations, responsive CSS, and accessibility labels.
17. Run backend tests, frontend build/lint, health checks, and manual role isolation checks.
```

A practical vertical slice

Start with profiles -> sign-up/sign-in -> profile/me -> one patient dashboard. Then add clinician directory -> availability -> booking. Add clinician appointment status -> evaluation. Add medical intake -> authorized clinical profile. Finally add each assessment region and admin management. This order follows the code dependencies and provides a working demonstration early.

16. Final Architecture Summary

The complete system can be recalled as a set of stable boundaries.

```
Browser
  React + React Router + i18next
  AuthProvider / RoleRoute
  Pages -> shared components -> Axios
       | bearer token
       v
Express
  CORS + parsers
  Router -> requireAuth -> requireRole
  Controller -> service -> Supabase client
       | service key only on backend
       v
Supabase
  Auth users
  PostgreSQL profiles, clinic, records, evaluations, assessments
  RLS policies + indexes + private Storage
       | JSON / signed URLs
       v
UI state, dashboards, calendars, clinical forms
```

- Starts at backend/src/server.js and frontend/src/main.jsx.
- The backend exposes role-scoped REST endpoints under /api.
- Supabase is both identity provider and PostgreSQL/Storage backend.
- Scheduling is computed from working hours minus appointments/time-off in UTC.
- Medical and clinical data is protected by server checks and RLS.
- The frontend is bilingual, responsive, and organized by role and clinical region.
- Tests encode date, booking, isolation, validation, evaluation, and medical-record invariants.

Presentation closing sentence: ?This project separates browser responsibilities, HTTP orchestration, domain rules, and database enforcement so that a patient-facing clinic workflow remains understandable, testable, and protected at every boundary.?

Appendix: Important File Inventory

This appendix names the authored files found in the repository and gives each one a concise responsibility. It is intentionally compact; generated dependencies are excluded.

backend/src/server.js: Loads dotenv, dynamically imports the app, and listens on PORT.

backend/src/app.js: Creates Express app, CORS/body parsing, mounts routers, 404 response.

backend/src/config/supabase.js: Validates env and creates public/admin Supabase clients.

backend/src/middleware/requireAuth.js: Bearer token validation, profile reload, active account gate.

backend/src/middleware/requireRole.js: Allowed-role authorization middleware.

backend/src/utils/http.js: API errors, success/error envelopes, required and pick helpers.

backend/src/controllers/*.js: HTTP handlers for auth, admin, clinic, clinician, records, assessments.

backend/src/services/authService.js: Supabase sign-up/sign-in and profile fallback logic.

backend/src/services/availabilityService.js: UTC date validation, slot generation, grouping and shuffle.

backend/src/services/*AssessmentService.js: Region-specific validation and schema boundaries.